

SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense

Editable student template for application-based cloud virtualization and SDDC/cloud operations defense



Assessment Metadata

Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense
Submission Type	Individual digital case analysis and infrastructure defense based on the same capstone title used in CO1, CO2 AT1 and CO2 AT2
Faculty	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Submission Mode	Completed DOCX template and GitHub repository link through Google Classroom

How This Template Works

- Use the same capstone title used in CO1, CO2 AT1 and CO2 AT2.
- Do not recalculate all VM sizing or repeat all configuration screenshots. Summarize the final AT1/AT2 evidence and use it for design defense.
- Answer all five questions with application-oriented reasoning connected to the capstone product.
- Include sustainable cloud operation decisions wherever relevant: right-sized resources, log retention, storage lifecycle, scheduled shutdown, API-call reduction and VM/container/serverless comparison.
- Paste the final topology diagram in the given response area. The sample diagram is only a structure; redraw it for the student's own product.
- Upload this completed document and supporting evidence to GitHub, then submit the repository link through Google Classroom.

Assessment Focus

AT1 answered what VM is required. AT2 showed configuration or simulation evidence. AT3 asks whether the student can defend the complete infrastructure design under application, security, operations and failure conditions.

Student and Capstone Details

Student Name	Konatham Mahesh
Register Number	192311014
Class / Slot	Slot B

Capstone Title	SafeWalk – Cloud Route Intelligence Using Crowd Reports and Risk Heatmaps
CO1 Evidence Link	[Insert CO1 concept map / presentation GitHub link here]
CO2 AT1 VM Recommendation	vCPU: 4 RAM: 24 GB Storage: 50 GB (app disk) + ~110 GB object storage VM Class: Two-tier (API VM + managed PostgreSQL/PostGIS)
CO2 AT2 Evidence Link	[Insert CO2 AT2 evidence repository link here]
GitHub Repository Link	[Insert GitHub repository link here]
Submission Date	[Insert submission date]

Evidence Carry-Forward Summary

Evidence Source	Student Entry	How It Supports AT3 Defense
CO1 concept map / presentation	SafeWalk system architecture concept map from CO1 covering user roles (Commuter/Reporter, Moderator, Admin) and core modules (reporting, heatmap, routing, alerts, admin dashboard).	Confirms the module list and user roles carried into the Q3 topology table and the Q4 access-control design.
CO2 AT1 sizing workbook	CO2 AT1 workload classification (database-heavy + analytics/AI), traffic calibration (~20 RPS peak, 18,000 daily requests) and CPU/RAM/storage sizing (4 vCPU, 24 GB RAM, 50 GB disk).	Provides the exact sizing numbers defended in Q1 (virtualization value) and Q5 (right-sizing/Green Computing).
VM calibration sheet	VM calibration sheet: CPU score 9 (4-8 vCPU band), RAM buildup to ~24 GB, storage split between 50 GB app disk and ~110 GB object storage for photos.	Justifies the two-tier VM plan and the decision to keep uploads off the VM disk, referenced in Q1 and Q5.
CO2 AT2 practical evidence	Azure Ubuntu Server 24.04 LTS lab VM (B2as_v2, SSH-key auth, port 22 only); Docker NGINX container (port 8080) with verified logs; VirtualBox local VM with snapshot 'before-update-or-demo'.	Confirms hands-on VM/container configuration, SSH-only security posture and snapshot-based rollback referenced in Q2 and Q5.
Capstone architecture decision	Two-tier pilot architecture: API VM + managed PostgreSQL/PostGIS, risk-scoring/heatmap job as a separate container, notification dispatch as a serverless function, object storage for photos.	Forms the baseline design defended in the Q3 topology and the Q4 deployment-alternative comparison.

Q1: Virtualization Value and Capstone Cloud Deployment Summary

Explain your capstone product, expected workload, cloud need and deployment context. Analyze how virtualization improves cost efficiency, resource utilization, workload isolation, infrastructure flexibility and cloud deployment readiness for your product.

Design Point	Student Response
Capstone product and users	SafeWalk is a cloud route-intelligence app for pedestrians and commuters (students, night-shift workers, delivery staff, women commuters) who need safer walking routes; it uses crowd-submitted incident reports and a risk heatmap. Campus/local safety authorities use it to monitor hotspot trends.
Expected workload	Database-heavy with an analytics/AI layer: frequent geospatial writes (incident reports) and reads (route/heatmap lookups), plus a continuously recalculating risk-scoring job. Pilot scale: ~3,000 registered users, ~600

	concurrent peak users, ~20 RPS at peak, ~18,000 requests/day.
Cloud need	An always-on backend API and geospatial database, object storage for incident photos, push notification delivery, and elastic compute to absorb morning/evening commute-hour traffic spikes.
Virtualization value	Virtualization lets the API, database and risk-scoring workloads run as isolated, independently right-sized instances on shared hardware, so each component can be resized, redeployed or rolled back without affecting the others.
Cost and utilization benefit	The calibrated 4 vCPU / 24 GB API VM avoids over-provisioning; commute-hour-only autoscaling avoids paying for idle peak-sized capacity for the rest of the day.
Isolation and flexibility benefit	Splitting the API VM, managed database and containerized risk-scoring service means a crash in the analytics job cannot take down the API, and each tier can be patched or scaled on its own schedule.

Virtualization is central to SafeWalk because the product bundles three very different workloads on one product idea: a request-driven API, a geospatial database, and a continuously running risk-scoring analytics job. Running these on one shared physical host through virtualization/containers means each workload gets an isolated, right-sized slice of compute instead of one oversized machine. The CO2 AT1 calibration produced a CPU score of 9, driven mainly by the geospatial database traffic and the analytics/AI risk-scoring module, which is why the plan splits the analytics job into its own 2 vCPU/6-8 GB container rather than growing the API VM further. Virtualization also gives SafeWalk cost efficiency: the 4 vCPU/24 GB API VM matches the ~20 RPS peak calculated from 3,000 registered users, so the team is not paying for unused capacity. Resource utilization improves because the managed PostgreSQL+PostGIS database and object storage absorb the heaviest read/write and file workloads instead of competing with the API for the same disk and memory. Workload isolation is achieved by keeping the stateful API/DB tier on VM/managed-service infrastructure while the stateless risk-scoring, dashboard and notification pieces run as containers or serverless functions. Infrastructure flexibility comes from this same separation: the risk-scoring container can be rescheduled or scaled independently of the API during traffic spikes. Finally, cloud deployment readiness is supported by the pilot-to-production growth path already planned in AT1 -- starting as a two-tier design and evolving into a three-tier design with a load balancer as usage grows -- which virtualization makes possible without a redesign.

Q2: Rapid Provisioning, Testing and Virtual Machine Lifecycle Case

Using your CO2 AT1 VM recommendation and CO2 AT2 practical evidence, explain how virtual machines, VM images, templates, snapshots, clones or containers support rapid provisioning, testing, deployment, rollback and maintenance for your capstone product.

Lifecycle Element	AT1/AT2 Evidence Used	Capstone Use / Justification
VM image or base environment	Ubuntu Server 24.04 LTS image selected and provisioned in AT1/AT2 (Azure Marketplace image and local VirtualBox ISO).	A standard base image gives the API VM a consistent OS/security baseline across local development, pilot and production.
Template or repeatable setup	Azure VM created with a repeatable naming/config pattern (resource group csa15-rg,	A reusable VM template lets the API tier be redeployed identically for scaling out or disaster

	VM size Standard B2as_v2) from AT2.	recovery.
Snapshot / rollback	VirtualBox snapshot 'before-update-or-demo' created 30-07-2026, retained until the update/demo was confirmed stable.	The same before-deployment/before-migration snapshot pattern from AT1 protects the production API VM and database schema changes with fast rollback.
Clone / testing copy	Clone strategy documented in AT1 Activity 9 (clone for isolated risk-scoring algorithm testing and team demo copies); not executed in AT2 due to local storage limits.	A cloned VM lets a new risk-scoring algorithm be tested in isolation before it is promoted to the production container.
Container or VM decision	Docker NGINX container tested and verified in AT2 (docker run, port 8080 mapping, docker logs, clean stop/remove).	Confirms containers are viable for SafeWalk's stateless services; the risk-scoring job, dashboard and notification worker are containerized/serverless while the API stays on VM and the database stays managed.
Maintenance and update plan	Snapshot-before-patch pattern and monthly OS/package update cycle from AT1 Activity 9.	Monthly patching is preceded by a snapshot so a broken update can be rolled back without downtime to the API.

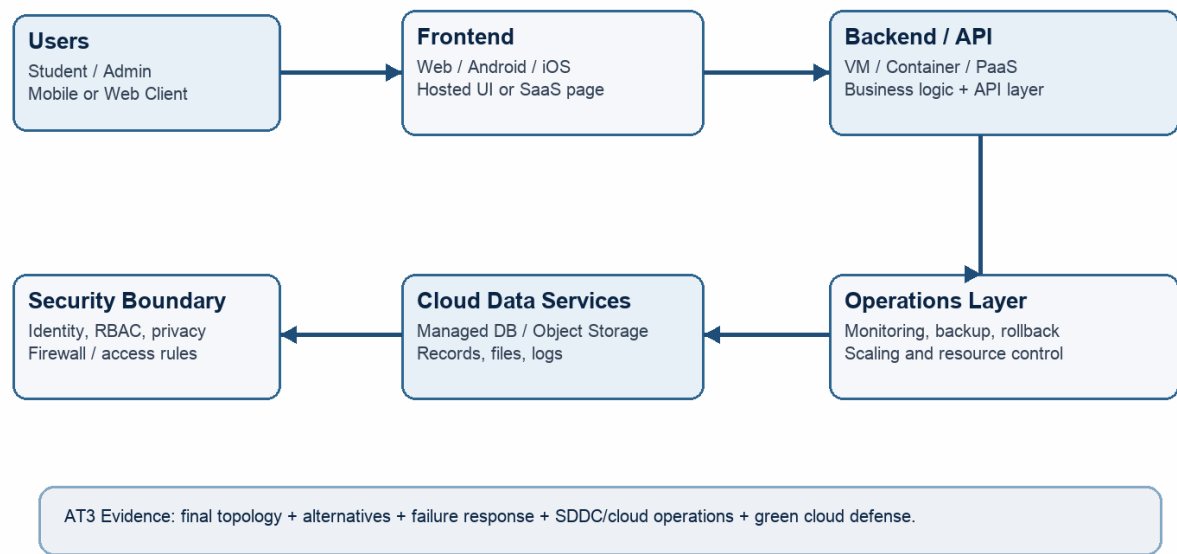
Using a known base image (Ubuntu Server 24.04 LTS) and a repeatable VM configuration removes the guesswork of manual setup, so the API VM can be recreated in minutes for scaling, recovery or a fresh pilot environment instead of hours of manual installation. Snapshots taken before every deployment, schema migration or OS patch mean a failed change can be rolled back almost immediately rather than triggering a lengthy manual rebuild, which directly reduces both deployment delay and downtime risk. Clones give an isolated space to test a new risk-scoring algorithm or a schema change without touching the live pilot environment, so mistakes are caught before they reach real users. Containerizing the stateless pieces (risk-scoring job, dashboard, notification worker) means these components can be redeployed independently and far faster than a full VM, since only the container image changes rather than the whole OS state. Together, these lifecycle practices mean most SafeWalk changes are low-risk, reversible and fast, which keeps deployment delay and rollback risk low even as the system grows from prototype to pilot to production.

Q3: Final Cloud Deployment Topology and SOA/API Service Integration Case

Draw and explain the final deployment topology for your capstone product. Show frontend, backend/API, database, storage, authentication, notification, analytics and any VM/container/managed cloud service components. Explain how SOA/API-based integration connects these modules and supports modular cloud service design.

Capstone Cloud Deployment Topology - Sample Structure

Students should redraw this with their own modules, services, VM/container choices and security boundaries.



Architecture Component	Technology / Service	VM / Container / Managed Service / SaaS	Integration Method
Frontend	React (web) / hybrid Android client	Managed hosting / CDN	HTTPS calls to backend REST API
Backend/API	Node.js/Express REST API	Container (Docker) on API VM	Exposes JWT-secured REST endpoints for reports, routes and admin data
Database	PostgreSQL + PostGIS	Managed database service	API connects over a private/secured DB connection string; geospatial queries via PostGIS
Storage	Object storage (blob/S3-compatible)	Managed service	Backend issues signed upload/download URLs for incident photos
Authentication	JWT-based authentication	Runs inside the backend API container	Issues/validates tokens for Commuter, Moderator and Admin roles on each request
Notification	Push notification service	Serverless function	Triggered on new nearby-incident reports; calls the push notification API
Analytics / AI module	Python risk-scoring / heatmap clustering module	Container (independently scalable)	Reads report data via internal API/queue, writes updated risk scores back to the DB/cache

Paste or draw the final capstone deployment topology diagram here.

Each SafeWalk module -- frontend, backend/API, database, storage, authentication, notification and the analytics/AI module -- is exposed or consumed as a discrete service rather than being bundled into one monolith. The frontend never talks to the database or object storage directly; it always goes through the backend/API, which enforces authentication and role checks before forwarding a request. The risk-scoring/heatmap module communicates with the API asynchronously (via an internal API call or lightweight queue) instead of running inline in the request path, so a slow analytics recalculation never blocks a user's route request. Notifications are dispatched through a serverless function that is invoked by the API/queue rather than being part of the always-on API process. This SOA/API-based design is useful because it lets each module be developed, deployed, scaled and secured independently -- the database can be resized without touching the API code, the risk-scoring container can be redeployed without restarting the frontend, and a new client (e.g., a future iOS app) can reuse the same API without any backend changes.

Q4: Design Alternatives, Identity, Access and Federated Cloud Privacy Case

Compare at least two deployment alternatives for your capstone product, such as single VM, multi-tier VM, containerized deployment, managed backend, federated cloud or hybrid cloud model. Justify the selected design by analyzing identity management, role-based access control, federated access, data sharing, privacy protection and security risks.

Deployment Alternative	Advantages	Limitations / Risks	Identity and Privacy Impact
Alternative 1	Simple to deploy and manage; lowest infrastructure cost and complexity for an early prototype.	No isolation between API, database and analytics; single point of failure; cannot scale the heavy geospatial/analytics load independently.	All authentication and sensitive report/location data live on one instance, so a single compromise exposes everything.
Alternative 2	API, database and analytics scale and fail independently; managed database removes patching/backup/replication overhead; containers give fast, portable redeploys.	More components to secure, monitor and connect; slightly higher operating cost and configuration complexity than a single VM.	Each tier enforces its own access boundary (network rules, RBAC per service), which reduces the blast radius of any single compromise.
Selected design	Two-tier pilot (API VM + managed PostgreSQL/PostGIS, risk-scoring container, serverless notifications), growing into a three-tier design (frontend/API/analytics fully separated behind a load balancer)	Requires disciplined network and IAM configuration between tiers as the system grows; more moving parts than Alternative 1.	RBAC enforced at the API layer for Commuter/Moderator/Admin roles; sensitive location and photo data isolated to the managed database and access-controlled object storage, reducing exposure

	in production.		versus a single-VM design.
Security Area		Student Decision	
User roles	Commuter/Reporter, Verified Moderator, Admin (defined in CO1/AT1).		
Authentication method	JWT-based authentication for the application; SSH public-key authentication (no passwords) for infrastructure access, as demonstrated in CO2 AT2.		
Role-based access control	Endpoint-level RBAC: Commuters submit/view reports and routes; Moderators review and validate flagged reports; Admins manage risk zones, moderate data and view analytics.		
Federated access / partner access	Not required for the pilot. Campus/local safety authorities are given a scoped, read-only Admin-level role rather than full federated access, keeping the trust boundary simple until partner integration is justified.		
Sensitive data handled	User account credentials, precise user location data, incident report content, and uploaded incident photos.		
Privacy protection decision	Location and report data are encrypted in transit (HTTPS) and at rest in the managed database; photos are stored in access-controlled object storage using signed, time-limited URLs rather than public buckets; no credentials or tokens are stored in the repository (matching the AT2 assurance).		
The selected two-tier design (API VM + managed PostgreSQL/PostGIS, moving toward a three-tier production model) is better for identity, access and privacy than a single all-in-one VM because each tier can enforce its own boundary instead of relying on one shared surface. Role-based access control at the API layer means a Commuter account can never reach Moderator or Admin endpoints even if the frontend is compromised, since checks happen server-side on every request rather than being trusted from the client. Keeping the database managed and separate from the API VM means a compromise of the API process does not automatically grant direct database access -- the attacker still has to cross the managed-service network boundary. SSH key-only access to infrastructure (verified in AT2, no password authentication, port 22 restricted) removes the most common credential-leakage vector for the VM itself. Because sensitive data -- precise locations, report content, photos -- is concentrated in the managed database and access-controlled object storage rather than spread across VM disks, privacy protections (encryption, signed URLs) can be applied once at that boundary and still cover the whole system.			

Q5: Software-Defined Datacenter and Cloud Operations Defense Case

Analyze how your infrastructure will respond to operational and failure scenarios such as VM failure, database growth, traffic spike, credential leakage, service outage, resource contention or cost increase. Explain how SDDC concepts and cloud operations practices such as automated provisioning, software-defined networking, software-defined storage, monitoring, logging, backup, rollback, scaling, resource control and maintenance improve the final infrastructure design. Include Green Computing decisions such as resource right-sizing, storage lifecycle control, log retention, API-call reduction, scheduled shutdown, VM/container/serverless comparison and carbon/energy proxy indicators where relevant.

Operational / Failure Scenario	Expected Impact	SDDC / Cloud Operations Response	Evidence or Control
VM failure	API becomes unreachable; users cannot view the heatmap or get route	Automated VM health checks trigger a restart; production moves to an	Azure VM stop/start behavior verified in AT2; planned health-probe and

	recommendations.	autoscaling group behind a load balancer so a single VM failure does not take down the service.	autoscaling configuration.
Database growth	Slower geospatial queries and rising storage cost as incident reports accumulate.	Managed database auto-scaling storage, indexing on location/timestamp fields, periodic archiving of old/decayed reports.	PostGIS schema and record-growth estimate from AT1 Activity 6 (storage calibration).
Traffic spike	Morning/evening commute-hour peak (~20 RPS, 600 concurrent users) causes latency or dropped requests.	Peak-window-only autoscaling group for the API tier, as planned in AT1, so extra capacity is added only when traffic actually spikes.	Traffic calibration (concurrent users, RPS) from AT1 Activity 3.
Credential leakage	Unauthorized access to user accounts or backend infrastructure.	SSH key-only VM access, no plaintext credentials in the repository, JWT token expiry/rotation at the application layer.	AT2 evidence: Azure VM configured with SSH public-key auth only, port 22 restricted, no passwords recorded or shared.
Service outage	App/API becomes fully unreachable for all users.	Snapshot-based rollback to the last known-good state, redeploy the API container/VM image, verify via status/monitoring checks.	VirtualBox snapshot 'before-update-or-demo' evidence from AT2; snapshot/clone plan from AT1 Activity 9.
Resource contention	Risk-scoring/heatmap job competes with the API for CPU/RAM on the same instance.	Risk-scoring runs as its own container (2 vCPU, 6-8 GB RAM), fully decoupled from the API VM.	AT1 CPU score of 9 (Activity 4) specifically justified separating the analytics workload into its own service.
Cost increase	Running the API VM and database at full/idle-peak size around the clock raises cloud spend.	Scheduled shutdown of non-production/lab VMs when idle, and a right-sized (not over-provisioned) VM instead of a larger default size.	AT2 evidence: VM stopped/deallocated after each lab session; AT1 right-sizing calculation (CPU score, RAM buildup).

Operational Readiness Area		Student Plan
Automated provisioning		VM/container creation follows a repeatable naming and configuration pattern (resource group + VM size fixed in AT2) so environments can be rebuilt on demand instead of manually.
Software-defined networking		Inbound network rules restricted to SSH (22) and, only during testing, HTTP (80) — verified and reviewed in AT2's networking closeout step.
Software-defined storage		Managed database storage and managed object storage abstract away physical disk management, so storage grows without manual disk resizing on the VM.
Monitoring and logging		VM/container health checks (docker logs, systemctl status, uname/free/df from AT2) plus centralized application logging with a bounded retention window.
Backup and rollback		Pre-deployment/pre-migration snapshots retained only as long as rollback protection is needed, plus automated backups on the managed database.
Scaling and elasticity		Peak-window autoscaling for the API tier during commute hours; the risk-scoring container scales or reschedules independently of the API.
Resource control and maintenance		Monthly patch cycle preceded by a snapshot; idle VMs stopped/deallocated between sessions to limit unnecessary resource use.
Green Computing and sustainability control		Right-sized VM instead of an oversized default, scheduled shutdown of idle resources, bounded log retention, and container/serverless choices for lightweight or event-driven components.
Green Cloud Decision		Student Justification

VM right-sizing from CO2 AT1	AT1's CPU/RAM calibration (score 9, ~18 GB rounded to 24 GB) avoided jumping straight to a large default VM size; the analytics workload was offloaded to its own container instead of inflating the API VM further.
VM vs container vs serverless choice	Stateful API and database stay on VM/managed service; stateless risk-scoring and dashboard are containerized; event-driven notifications run serverless -- each workload uses the lightest option that actually fits it, avoiding always-on capacity for infrequent tasks.
Scheduled shutdown / idle-resource control	Non-production/lab VMs are stopped or deallocated after each session, demonstrated directly in AT2's closeout evidence; production autoscaling avoids paying for idle peak-sized capacity outside commute hours.
Log and file retention policy	Application logs retained for a bounded window (~90 days, per AT1's storage estimate) instead of indefinitely, keeping log storage growth in check.
Storage lifecycle and backup cleanup	Old incident photos and stale cached heatmap tiles are moved to lower-cost storage tiers or purged after a retention window; VM snapshots are deleted once rollback protection is no longer needed rather than kept indefinitely.
API-call or polling reduction	Client refresh interval is capped (about once per minute) instead of continuous polling, and hot heatmap tiles are cached in Redis to avoid recomputing the same risk scores repeatedly.
Carbon/energy proxy indicator to track	VM/container CPU-hours and percentage idle time, tracked as a simple proxy for energy use, with the goal of reducing idle-hour percentage after scheduled shutdown and autoscaling are enabled.

The current SafeWalk infrastructure is ready for pilot deployment but not yet production. The two-tier design (4 vCPU/24 GB API VM, managed PostgreSQL+PostGIS, ~110 GB object storage for photos, a separate risk-scoring container and serverless notifications) is backed by concrete AT1 sizing calculations and AT2 practical evidence, including a working Azure VM, a tested Docker container and a snapshot-based rollback plan. Reliability is reasonable for a pilot -- snapshots before every deployment/patch and a documented clone strategy give a real rollback path -- but production needs an actual load balancer and autoscaling group rather than a single API VM, since commute-hour traffic (~20 RPS peak) can still overwhelm one instance during a spike. Security is solid at the infrastructure level (SSH key-only access, restricted inbound ports, no credentials in the repository) but the application-level JWT/RBAC enforcement and audit logging still need to be implemented and tested end-to-end before production use. SDDC/cloud-operations maturity is currently manual-but-documented (snapshots, patch cycles, shutdown steps done by hand); before production this should move to automated provisioning, monitoring dashboards and automated backups rather than checklist-driven steps. Green Computing decisions -- right-sizing, scheduled shutdown, log retention, and container/serverless choices for lightweight workloads -- are already part of the design and should be carried forward as enforced policy (not just a one-time decision) once the system moves to production scale.

CO2 Reflection and Individual Defense

This section must be individually written. It should show what the student understands from the complete CO2 chain: infrastructure sizing, practical configuration evidence and final operations defense.

Reflection Prompt	Student Response
What is the strongest infrastructure decision in your capstone design?	Separating the risk-scoring/analytics workload and the database from the core API VM from day one -- it keeps the API lean while isolating the two heaviest, most failure-prone workloads.
Which CO2 concept became clearer after AT1, AT2 and AT3?	How to translate workload signals (traffic, database writes, analytics load) into a defensible vCPU/RAM score using AT1's scoring method, instead of guessing a VM size.
Which risk is most serious for your capstone: failure, cost, privacy, scaling or sustainability?	Scaling risk is the most serious for SafeWalk, since usage is inherently bursty around commute-hour peaks; cost and sustainability are the second concern if idle resources aren't controlled.
What evidence would you show during viva to defend your infrastructure design?	AT1's traffic/CPU/RAM calibration tables, AT2's Azure VM and Docker container screenshots (with SSH-only access and snapshot evidence), and this AT3 topology and failure-response tables.
What will you improve before final capstone deployment?	Move from a single API VM to a real three-tier design with a load balancer, implement automated backups and a monitoring dashboard, and turn the scheduled-shutdown/log-retention policies into actual automation rather than manual steps.

Student-Facing Assessment Criteria

Criteria	Descriptor
Capstone continuity	The answer uses the same capstone title and connects CO1, CO2 AT1 and CO2 AT2 evidence.
Virtualization and lifecycle reasoning	VM, image, snapshot, clone, container or managed-service decisions are technically justified.
Topology and integration clarity	Architecture diagram and SOA/API explanation show clear module interaction.
Identity, access and privacy	Roles, authentication, RBAC, federated access, sensitive data and privacy decisions are defended.
SDDC/cloud operations	Failure response, monitoring, logging, scaling, backup, rollback and resource control are addressed.
Green Computing defense	Right-sizing, retention, API efficiency, idle-resource control and sustainable cloud choices are explained.
Submission quality	GitHub, template completion, evidence references and individual reflection are complete.

GitHub and Google Classroom Submission

Repository Item	Expected Content
README.md	Capstone title, student details, final VM recommendation, deployment topology summary and AT3 evidence index.
Completed AT3 document	This completed editable DOCX or exported PDF if instructed.
Diagram image	Final topology diagram if created separately.
Evidence references	Links or screenshots from CO1, CO2 AT1 and CO2 AT2 as needed.
Green Computing note	Short README section explaining sustainable cloud choices and resource-control decisions.
Google Classroom	Submit the GitHub repository link through GCR.

Student Assurance

I confirm that this case analysis, design reasoning, topology diagram and final defense statement were prepared individually by me for the stated capstone title. I have not uploaded passwords, tokens, private keys or sensitive account information.

Student Name	Konatham Mahesh
Register Number	192311014
Signature	maheshk
Date	[Insert submission date]

Faculty Verification

Faculty Name	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Constructive Feedback / Remarks	
Strength Observed	
Improvement Suggested	
Signature / Date	